

BÀI 10. LÀM VIỆC VỚI TỆP (FILES) & JSON/CSV/SQL CƠ BẢN

10.1. Thao tác với Tệp văn bản

10.1.1. Khái niệm về tệp và tầm quan trọng trong lập trình ứng dụng

Một **tệp (file)** là một tập hợp các dữ liệu được lưu trữ trên một thiết bị lưu trữ (như ổ cứng, USB, SSD) dưới một tên cụ thể. Tệp cho phép lưu trữ dữ liệu một cách **đai đẳng (persistent)**, nghĩa là dữ liệu vẫn tồn tại ngay cả khi chương trình kết thúc.

10.1.2. Tầm quan trọng trong lập trình ứng dụng:

- **Lưu trữ dữ liệu lâu dài:** Chương trình có thể ghi dữ liệu ra tệp để sử dụng lại sau này (ví dụ: cài đặt người dùng, nhật ký lỗi, dữ liệu cấu hình).
- **Chia sẻ dữ liệu:** Tệp là cách phổ biến để trao đổi dữ liệu giữa các chương trình hoặc giữa các hệ thống khác nhau.
- **Đọc dữ liệu đầu vào:** Chương trình có thể đọc dữ liệu từ tệp thay vì yêu cầu người dùng nhập liệu mỗi lần chạy.
- **Quản lý cấu hình:** Nhiều ứng dụng lưu trữ cấu hình trong tệp văn bản.
- **Ghi nhật ký (Logging):** Ghi lại các sự kiện, lỗi, hoặc hoạt động của chương trình vào tệp log.

10.1.3. Các chế độ mở tệp: 'r' (read), 'w' (write), 'a' (append), 'x' (create), 'b' (binary)

Khi bạn làm việc với tệp trong Python, bạn cần "mở" tệp trước, chỉ định tên tệp và **chế độ (mode)** bạn muốn mở tệp.

Chế độ	Ý nghĩa	Ghi chú
'r'	Đọc (read) - Mở tệp để đọc.	Mặc định. Nếu tệp không tồn tại, sẽ gây lỗi <code>FileNotFoundError</code> .
'w'	Ghi (write) - Mở tệp để ghi.	Nếu tệp đã tồn tại, nội dung hiện có sẽ bị xóa sạch . Nếu tệp không tồn tại, một tệp mới sẽ được tạo.

'a'	Thêm vào cuối (append) - Mở tệp để ghi.	Dữ liệu mới sẽ được thêm vào cuối tệp. Nếu tệp không tồn tại, một tệp mới sẽ được tạo.
'x'	Tạo mới (create) - Mở tệp để ghi.	Chỉ tạo tệp mới. Nếu tệp đã tồn tại, sẽ gây lỗi <code>FileExistsError</code> .
'b'	Chế độ nhị phân (binary) - Sử dụng với các chế độ khác (ví dụ: 'rb', 'wb', 'ab').	Dùng cho các tệp không phải văn bản (hình ảnh, video, tệp nén). Không thể đọc/ghi chuỗi trực tiếp mà phải là byte.
'+'	Đọc và Ghi (read and write) - Sử dụng với các chế độ khác (ví dụ: 'r+', 'w+', 'a+').	Cho phép cả đọc và ghi trên cùng một tệp. r+ không xóa nội dung, w+ xóa nội dung, a+ thêm vào cuối và có thể đọc.

10.1.4. Hàm `open()`, phương thức `read()`, `readline()`, `readlines()`, `write()`, `writelines()`

Hàm `open()`: Đây là hàm dùng để mở một tệp. Nó trả về một đối tượng tệp (file object) mà bạn có thể dùng để đọc hoặc ghi dữ liệu.

Cú pháp: `file_object = open(file_path, mode, encoding='utf-8')`

- `file_path`: Đường dẫn đến tệp (ví dụ: "my_file.txt", "data/report.log").
- `mode`: Chế độ mở tệp (ví dụ: 'r', 'w', 'a').
- `encoding` (tùy chọn): Mã hóa ký tự (rất quan trọng cho tệp văn bản, đặc biệt là tiếng Việt, thường dùng 'utf-8').

Các phương thức đọc:

- **`read(size=-1)`:** Đọc toàn bộ nội dung của tệp dưới dạng một chuỗi duy nhất. Nếu size được chỉ định, nó sẽ đọc tối đa size byte/ký tự.

```
with open("example.txt", "r", encoding="utf-8") as f:
    content = f.read()
    print(content)
```

- **`readline()`:** Đọc một dòng duy nhất từ tệp. Bao gồm ký tự xuống dòng `\n` ở cuối dòng.

```
with open("example.txt", "r", encoding="utf-8") as f:
    first_line = f.readline()
    second_line = f.readline()
    print(first_line, end="") # Dùng end="" để tránh in thêm dòng trống
    print(second_line, end="")
```

- **readlines():** Đọc tất cả các dòng của tệp và trả về chúng dưới dạng một **List các chuỗi**, với mỗi chuỗi là một dòng (bao gồm \n).

```
with open("example.txt", "r", encoding="utf-8") as f:
    all_lines = f.readlines()
    for line in all_lines:
        print(line, end="")
```

Các phương thức ghi:

- **write(string):** Ghi một chuỗi vào tệp. Nó không tự động thêm ký tự xuống dòng, bạn phải thêm \n nếu muốn xuống dòng. Trả về số ký tự đã ghi.

```
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Đây là dòng đầu tiên.\n")
    f.write("Đây là dòng thứ hai.")
```

- **writelines(list_of_strings):** Ghi một List các chuỗi vào tệp. Mỗi chuỗi trong List sẽ được ghi liên tiếp, bạn phải tự đảm bảo các ký tự xuống dòng \n nếu muốn mỗi chuỗi là một dòng riêng biệt.

```
lines_to_write = ["Dòng 1 trong writelines\n", "Dòng 2 trong\n", "Dòng cuối."]
with open("output.txt", "a", encoding="utf-8") as f: # Mở chế độ 'a' để
    thêm vào cuối
    f.writelines(lines_to_write)
```

10.1.5. Đóng tệp bằng close() hoặc sử dụng with open()

Sau khi thao tác với tệp xong, bạn **phải đóng tệp** để:

- Giải phóng tài nguyên hệ thống.
- Đảm bảo tất cả dữ liệu đã được ghi vào tệp (đôi khi dữ liệu được đệm trong bộ nhớ và chỉ được ghi ra đĩa khi tệp được đóng).

1. **Sử dụng close():** Bạn phải gọi phương thức close() một cách tường minh.

```
f = open("my_file.txt", "w")
try:
    f.write("Hello world!")
finally: # Đảm bảo close được gọi ngay cả khi có lỗi
    f.close()
```

Cách này dễ bị lỗi nếu bạn quên gọi close() hoặc nếu có lỗi xảy ra trước khi close() được gọi.

2. **Sử dụng with open() (cách được khuyến nghị):** Đây là cách tốt nhất để làm việc với tệp trong Python. Nó đảm bảo tệp sẽ tự động được đóng ngay cả khi có lỗi xảy ra. with statement quản lý "context" (ngữ cảnh) của tệp.

```
with open("my_file.txt", "w", encoding="utf-8") as f:
    f.write("Hello from with open!")
    # File tự động đóng khi thoát khỏi khối 'with'
```

Khi khối with kết thúc (dù là bình thường hay do lỗi), phương thức `__exit__` của đối tượng tệp sẽ được gọi, và nó sẽ tự động đóng tệp cho bạn.

10.2. Đọc/Ghi dữ liệu JSON & CSV

10.2.1. JSON (JavaScript Object Notation): định dạng dữ liệu phổ biến trong web. Cú pháp, ưu điểm.

JSON (JavaScript Object Notation) là một định dạng dữ liệu nhẹ, dễ đọc cho con người và dễ phân tích cho máy tính. Nó được sử dụng rộng rãi để trao đổi dữ liệu giữa máy chủ và ứng dụng web, cũng như để lưu trữ cấu hình.

Cú pháp: JSON dựa trên hai cấu trúc chính:

1. **Đối tượng (Object):** Tập hợp không có thứ tự của các cặp tên/giá trị. Trong Python, nó tương ứng với **Dictionary**.
 - Cú pháp: { "key1": value1, "key2": value2 }
 - Tên (key) phải là chuỗi và nằm trong dấu ngoặc kép.
 - Giá trị (value) có thể là chuỗi, số, boolean (true/false), null, đối tượng khác, hoặc mảng.
2. **Mảng (Array):** Tập hợp có thứ tự của các giá trị. Trong Python, nó tương ứng với **List**.
 - Cú pháp: [value1, value2, value3]

Ví dụ JSON:

```
#JSON
{
  "name": "Bim",
  "age": 30,
  "isStudent": false,
  "courses": ["Math", "Physics"],
  "address": {
    "street": "123 HTK St",
```

```
    "city": "Anytown"
  },
  "phoneNumbers": [
    { "type": "home", "number": "123-456-7890" },
    { "type": "work", "number": "098-765-4321" }
  ],
  "nullValue": null
}
```

Ưu điểm của JSON:

- **Dễ đọc và viết:** Cú pháp đơn giản, giống với cách biểu diễn đối tượng trong nhiều ngôn ngữ lập trình.
- **Nhẹ:** Cấu trúc gọn gàng, ít "dư thừa" so với XML.
- **Hỗ trợ đa ngôn ngữ:** Hầu hết các ngôn ngữ lập trình hiện đại đều có thư viện để làm việc với JSON.
- **Phổ biến:** Là định dạng chuẩn cho API web.

Module json trong Python: `json.load()`, `json.dump()`

Python có một module tích hợp sẵn là json để làm việc với dữ liệu JSON.

- **`json.dump(obj, fp, ...)`:** Ghi đối tượng Python (obj) vào tệp (fp) dưới định dạng JSON.
 - o obj: Đối tượng Python (Dict, List, String, Int, Float, Bool, None) cần ghi.
 - o fp: Đối tượng tệp đã mở ở chế độ ghi.
 - o indent (tùy chọn): Số khoảng trắng để thụt lề, giúp tệp JSON dễ đọc hơn.
 - o ensure_ascii (tùy chọn): Mặc định là True. Nếu False, sẽ cho phép các ký tự non-ASCII được ghi trực tiếp (hữu ích cho tiếng Việt).

```
import json
data = {
    "name": "Bảo",
    "age": 25,
    "isEmployed": True,
    "hobbies": ["reading", "coding"]
}
with open("data.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=4, ensure_ascii=False)
# Output: data.json sẽ chứa dữ liệu JSON được định dạng đẹp
```

- **json.load(fp)**: Đọc dữ liệu từ tệp (fp) và phân tích nó thành đối tượng Python.
 - fp: Đối tượng tệp đã mở ở chế độ đọc.

```
import json

with open("data.json", "r", encoding="utf-8") as f:
    loaded_data = json.load(f)
print(loaded_data)
print(type(loaded_data)) # Output: <class 'dict'>
print(loaded_data["name"]) # Output: Bảo
```

- **json.dumps(obj, ...)**: Chuyển đổi đối tượng Python (obj) thành một **chuỗi JSON**. (s - string)

```
json_string = json.dumps(data, indent=2, ensure_ascii=False)
print(json_string)
```

- **json.loads(s)**: Chuyển đổi một **chuỗi JSON** (s) thành đối tượng Python. (s - string)

```
data_from_string = json.loads(json_string)
print(data_from_string["hobbies"]) # Output: ['reading', 'coding']
```

10.2.2. CSV (Comma Separated Values): định dạng bảng

CSV (Comma Separated Values) là một định dạng tệp văn bản đơn giản được sử dụng để lưu trữ dữ liệu dạng bảng (tabular data), nơi mỗi dòng trong tệp đại diện cho một hàng dữ liệu và các giá trị trong mỗi hàng được phân tách bằng một dấu phân cách (thường là dấu phẩy).

Cú pháp:

- Mỗi dòng là một bản ghi dữ liệu.
- Mỗi bản ghi bao gồm một hoặc nhiều trường, phân cách bởi dấu phẩy (hoặc dấu phân cách khác như dấu chấm phẩy, tab).
- Dòng đầu tiên thường là tiêu đề cột.

Ví dụ data.csv:

```
Code snippet
Name, Age, City
Bim, 13, HCM
Bo, 24, TG
Bin, 15, HN
```

Ưu điểm của CSV:

- **Đơn giản và phổ biến:** Dễ dàng tạo, đọc và chỉnh sửa bằng bất kỳ trình soạn thảo văn bản nào. Được hỗ trợ rộng rãi bởi các phần mềm bảng tính (Excel, Google Sheets).
- **Kích thước tệp nhỏ:** Dạng văn bản thuần túy, không có định dạng phức tạp.

Nhược điểm:

- Không có cấu trúc dữ liệu phức tạp (chỉ dạng bảng đơn giản).
- Xử lý dấu phẩy trong dữ liệu có thể phức tạp nếu không được bọc đúng cách (ví dụ: bằng dấu ngoặc kép).

Module csv trong Python (giới thiệu sơ lược)

Python có một module csv tích hợp sẵn, cung cấp các lớp và phương thức để đọc và ghi dữ liệu CSV. Nó xử lý tốt các trường hợp phức tạp như dấu phẩy trong dữ liệu, dấu ngoặc kép, v.v.

Đọc CSV:

```
import csv
with open('data.csv', 'r', encoding='utf-8') as file:
    csv_reader = csv.reader(file) # Tạo đối tượng đọc CSV
    header = next(csv_reader)     # Đọc dòng tiêu đề (nếu có)
    print(f"Tiêu đề: {header}")
    for row in csv_reader:        # Lặp qua từng hàng dữ liệu
        print(row) # row sẽ là một list các chuỗi: ['Bim', '13', 'HCM']
```

Ghi CSV:

```
Python
import csv
data_to_write = [
    ['Product', 'Price', 'Stock'],
    ['Laptop', 1200, 10],
    ['Mouse', 25.5, 50],
    ['Keyboard', 75, 30]
]
with open('products.csv', 'w', newline='', encoding='utf-8') as file:
    csv_writer = csv.writer(file) # Tạo đối tượng ghi CSV
    csv_writer.writerows(data_to_write) # Ghi nhiều hàng cùng lúc
# Hoặc: csv_writer.writerow(['New Item', 10, 5]) # Ghi từng hàng
```

- `newline=""` rất quan trọng khi mở tệp CSV để tránh các dòng trống thừa.

10.3. Cơ sở dữ liệu quan hệ (SQL) cơ bản

10.3.1. Khái niệm cơ sở dữ liệu và bảng (table), cột (column), hàng (row)

Cơ sở dữ liệu (Database - DB):

- Là một tập hợp có tổ chức của dữ liệu. Nó được thiết kế để lưu trữ, quản lý và truy xuất dữ liệu một cách hiệu quả.
- Mục tiêu chính là giúp lưu trữ dữ liệu một cách nhất quán, có cấu trúc và dễ dàng truy cập.

Cơ sở dữ liệu quan hệ (Relational Database):

- Là loại cơ sở dữ liệu phổ biến nhất, tổ chức dữ liệu thành các **bảng (tables)**.
- Các bảng này có thể được liên kết với nhau (quan hệ) thông qua các khóa (keys).

Các thành phần chính trong cơ sở dữ liệu quan hệ:

1. Bảng (Table):

- Giống như một bảng tính Excel. Mỗi bảng lưu trữ dữ liệu về một thực thể cụ thể (ví dụ: một bảng Customers lưu thông tin khách hàng, một bảng Products lưu thông tin sản phẩm).
- Mỗi bảng có một tên duy nhất.

2. Cột (Column) / Trường (Field):

- Mỗi cột trong bảng đại diện cho một thuộc tính hoặc một loại dữ liệu cụ thể của thực thể.
- Ví dụ: Trong bảng Customers, bạn có thể có các cột như CustomerID, Name, Email, Address.
- Mỗi cột có một tên và một kiểu dữ liệu (ví dụ: văn bản, số nguyên, ngày tháng).

3. Hàng (Row) / Bản ghi (Record):

- Mỗi hàng trong bảng đại diện cho một thể hiện duy nhất của thực thể.
- Ví dụ: Một hàng trong bảng Customers sẽ chứa toàn bộ thông tin của một khách hàng cụ thể.

Ví dụ cấu trúc bảng Students:

StudentID (PK)	Name	Age	Major	GPA
1	Bim	20	Computer Science	3.8

2	Bo	21	Electrical Eng.	3.5
3	Bìn	19	Business Admin.	3.2

- StudentID là **khóa chính (Primary Key - PK)**, một cột (hoặc tập hợp các cột) chứa các giá trị duy nhất để xác định mỗi hàng.

10.3.2. Giới thiệu SQL (Structured Query Language)

SQL (Structured Query Language) là ngôn ngữ chuẩn để quản lý và thao tác với cơ sở dữ liệu quan hệ. Nó không phải là một ngôn ngữ lập trình mục đích chung như Python, mà là một ngôn ngữ chuyên biệt để giao tiếp với database.

Bạn sử dụng SQL để:

- Truy vấn dữ liệu (lấy thông tin).
- Chèn dữ liệu mới.
- Cập nhật dữ liệu hiện có.
- Xóa dữ liệu.
- Tạo, sửa đổi và xóa cấu trúc cơ sở dữ liệu (bảng, chỉ mục, v.v.).

10.3.3. Các câu lệnh SQL cơ bản

Các câu lệnh SQL cơ bản: SELECT, INSERT, UPDATE, DELETE (chỉ giới thiệu cú pháp đơn giản, không đi sâu vào thực hành kết nối Python với DB trong tiết này)

Đây là các câu lệnh phổ biến nhất trong SQL, thuộc nhóm **DML (Data Manipulation Language)**.

1. **SELECT (Truy vấn dữ liệu):** Dùng để lấy dữ liệu từ một hoặc nhiều bảng.

- **Cú pháp cơ bản:** SELECT column1, column2, ... FROM table_name WHERE condition;
- Ví dụ:
 - Lấy tất cả cột và tất cả hàng từ bảng Students: SELECT * FROM Students;
 - Lấy tên và tuổi của tất cả sinh viên: SELECT Name, Age FROM Students;
 - Lấy tên của sinh viên có GPA trên 3.5: SELECT Name FROM Students WHERE GPA > 3.5;

2. **INSERT (Chèn dữ liệu):** Dùng để thêm một hàng mới vào bảng.

- **Cú pháp cơ bản:** INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);

- Ví dụ:

- Thêm một sinh viên mới vào bảng Students: INSERT INTO Students (StudentID, Name, Age, Major, GPA) VALUES (4, 'Diana', 20, 'Literature', 3.9);

3. **UPDATE (Cập nhật dữ liệu):** Dùng để sửa đổi dữ liệu hiện có trong một hoặc nhiều hàng của bảng.

- **Cú pháp cơ bản:** UPDATE table_name SET column1 = new_value1, column2 = new_value2, ... WHERE condition;

- Ví dụ:

- Cập nhật GPA của sinh viên có StudentID là 2 thành 3.7: UPDATE Students SET GPA = 3.7 WHERE StudentID = 2;
- Tăng tuổi của tất cả sinh viên khoa "Computer Science" lên 1: UPDATE Students SET Age = Age + 1 WHERE Major = 'Computer Science';

4. **DELETE (Xóa dữ liệu):** Dùng để xóa một hoặc nhiều hàng từ bảng.

- **Cú pháp cơ bản:** DELETE FROM table_name WHERE condition;

- Ví dụ:

- Xóa sinh viên có StudentID là 3: DELETE FROM Students WHERE StudentID = 3;
- Xóa tất cả sinh viên có tuổi dưới 20: DELETE FROM Students WHERE Age < 20;
- **Cẩn trọng:** DELETE FROM table_name; sẽ xóa **tất cả các hàng** trong bảng mà không có điều kiện.

BÀI TẬP:

Phần 1: Làm việc với Tập Văn bản (Files)

Bài tập	Yêu cầu Chi tiết	Thử thách Nâng cao
1. Ghi và	Viết chương trình: 1. Mở tệp log_data.txt ở chế độ ghi (w).	Sử dụng câu lệnh with open(...) as f: để đảm bảo tệp luôn được đóng, ngay cả khi xảy ra lỗi.

Đọc Tệp	2. Ghi 5 dòng văn bản bất kỳ (ví dụ: các thông báo log) vào tệp. 3. Đóng tệp. 4. Mở lại tệp ở chế độ đọc (r). 5. Đọc và in ra tất cả nội dung.	
2. Sao chép Tệp	Viết chương trình: 1. Đọc nội dung từ tệp input.txt (tự tạo nội dung). 2. Ghi nội dung đó vào một tệp mới tên là output_copy.txt.	Thay vì đọc và ghi toàn bộ tệp một lần (.read()), sử dụng vòng lặp for line in f: để đọc từng dòng và ghi từng dòng, mô phỏng việc xử lý tệp lớn.
3. Đếm Từ	Viết hàm nhận tên tệp văn bản. Hàm này phải đọc tệp và trả về tần suất xuất hiện của mỗi từ trong tệp dưới dạng Dictionary (bỏ qua chữ hoa/chữ thường và các ký tự đặc biệt).	Xử lý lỗi FileNotFoundError bằng try/except nếu người dùng nhập tên tệp không tồn tại.

Phần 2: Làm việc với Định dạng Dữ liệu (JSON & CSV)

Bài tập	Yêu cầu Chi tiết	Thử thách Nâng cao
1. Đọc và Thao tác với JSON	Tải và đọc tệp data.json mẫu sau: <pre>[{"name": "A", "score": 85}, {"name": "B", "score": 92}, {"name": "C", "score": 78}]</pre> Sử dụng thư viện json để: 1. Tải dữ liệu vào một List trong Python. 2. In ra tên của người có điểm cao nhất.	Viết code để thêm một trường mới ("status": "Passed") cho tất cả học viên có điểm , sau đó lưu lại kết quả vào một tệp JSON mới (updated_data.json).
2. Ghi Dữ liệu CSV	Viết chương trình: 1. Sử dụng thư viện csv. 2. Tạo một List of Lists (dữ liệu mẫu: tên, email, nghề nghiệp). 3.	Sử dụng csv.DictWriter thay vì csv.writer. Điều này yêu cầu bạn phải truyền dữ liệu là một List of Dictionaries và chỉ định fieldnames (tiêu đề cột).

	Ghi dữ liệu này vào tệp CSV tên contacts.csv.	
3. Phân tích Dữ liệu CSV	Sử dụng tệp CSV bạn vừa tạo (contacts.csv). 1. Đọc tệp CSV bằng csv.reader. 2. Dùng vòng lặp để đếm xem có bao nhiêu người trong danh sách có nghề nghiệp là 'Developer' (hoặc một nghề nào đó do bạn định nghĩa).	Nếu sử dụng csv.DictReader, hãy viết code để nhóm dữ liệu theo nghề nghiệp và đếm số lượng người trong mỗi nhóm.

Phần 3: Tương tác SQL Cơ bản (Sử dụng thư viện sqlite3)

- **Lưu ý:** Phần này giả định bạn có kiến thức cơ bản về SQL. Bạn sẽ sử dụng thư viện **sqlite3** có sẵn trong Python.

Bài tập	Yêu cầu Chi tiết	Thử thách Nâng cao
1. Tạo Bảng và Chèn Dữ liệu	1. Kết nối đến cơ sở dữ liệu (CSDL) inventory.db (nếu chưa có sẽ tự tạo). 2. Tạo một Bảng tên Products với các cột: id (INTEGER PRIMARY KEY), name (TEXT), price (REAL). 3. Chèn (INSERT) 3 bản ghi sản phẩm vào bảng. 4. commit() và đóng kết nối.	Sử dụng ? placeholders và phương thức .execute() hoặc .executemany() để chèn dữ liệu, tránh các vấn đề về SQL Injection.
2. Truy vấn và Cập nhật Dữ liệu	1. Kết nối lại CSDL inventory.db. 2. Truy vấn (SELECT) và in ra tên và giá của tất cả sản phẩm. 3. Cập nhật (UPDATE) giá của một sản phẩm bất kỳ thêm 10%. 4. Truy vấn lại và in ra để xác nhận thay đổi.	Truy vấn chỉ các sản phẩm có giá lớn hơn một ngưỡng nhất định. Sau đó, sử dụng câu lệnh DELETE để xóa một bản ghi thỏa mãn điều kiện.

THỰC HÀNH 10: LÀM VIỆC VỚI TỆP (FILE)

10.4. Đọc và ghi tệp văn bản

10.4.1. Viết chương trình đọc toàn bộ nội dung của một tệp văn bản

Mục tiêu: Thực hành đọc tệp cơ bản.

Bài tập: Tạo file `read_text_file.py`.

1. Tạo một tệp văn bản mới tên là `sample.txt` và ghi vài dòng văn bản vào đó bằng tay (hoặc bằng chương trình Python nếu muốn).
 - Nội dung mẫu:
 - Dòng đầu tiên.
 - Đây là dòng thứ hai.
 - Và đây là dòng cuối cùng của tệp.
2. Viết chương trình Python sử dụng `with open()` và `read()` để đọc toàn bộ nội dung của `sample.txt` và in ra màn hình.
3. Xử lý lỗi `FileNotFoundError` nếu tệp không tồn tại.

Gợi ý:

```
# read_text_file.py

print("--- ĐỌC TOÀN BỘ NỘI DUNG TỆP VĂN BẢN ---")

file_name = "sample.txt"

# Bước 1: Tạo file sample.txt nếu chưa có để test
try:
    with open(file_name, "w", encoding="utf-8") as f:
        f.write("Dòng đầu tiên.\n")
        f.write("Đây là dòng thứ hai.\n")
        f.write("Và đây là dòng cuối cùng của tệp.\n")
    print(f"Đã tạo/ghi đè file '{file_name}' để kiểm tra.")
except Exception as e:
    print(f"Không thể tạo file {file_name}: {e}")

# Bước 2: Đọc toàn bộ nội dung của sample.txt
try:
```

```
with open(file_name, "r", encoding="utf-8") as file:
    content = file.read()
    print(f"\nNội dung của tệp '{file_name}':")
    print(content)
except FileNotFoundError:
    print(f"Lỗi: Tệp '{file_name}' không tìm thấy.")
except Exception as e:
    print(f"Đã xảy ra lỗi khi đọc tệp: {e}")

print("\n--- ĐỌC TỪNG DÒNG VỚI readlines() ---")
try:
    with open(file_name, "r", encoding="utf-8") as file:
        lines = file.readlines()
        print(f"Đọc từng dòng từ tệp '{file_name}':")
        for i, line in enumerate(lines):
            print(f"Dòng {i+1}: {line.strip()}") # strip() để loại bỏ ký
            tự xuống dòng
except FileNotFoundError:
    print(f"Lỗi: Tệp '{file_name}' không tìm thấy.")
```

10.4.2. Ghi một chuỗi hoặc danh sách các dòng vào tệp mới

Mục tiêu: Thực hành ghi tệp cơ bản.

Bài tập: Tạo file write_text_file.py.

4. Ghi một chuỗi đơn lẻ ("Hello, Python File I/O!") vào một tệp mới tên là output.txt bằng chế độ 'w'.
5. Tạo một List các chuỗi, mỗi chuỗi đại diện cho một dòng văn bản. Ghi List này vào một tệp mới khác tên là my_notes.txt bằng chế độ 'w', đảm bảo mỗi chuỗi nằm trên một dòng riêng biệt.

Gợi ý:

```
# write_text_file.py
print("--- GHI CHUỖI VÀ DANH SÁCH DÒNG VÀO TỆP ---")

# 1. Ghi một chuỗi đơn lẻ vào output.txt
output_file_name = "output.txt"
single_string = "Hello, Python File I/O! This is the first line.\n" \
                "This is the second line written as part of the same
string."
```

```
try:
    with open(output_file_name, "w", encoding="utf-8") as file:
        file.write(single_string)
    print(f"Đã ghi chuỗi vào tệp '{output_file_name}'.")
except IOError as e:
    print(f"Lỗi khi ghi tệp '{output_file_name}': {e}")

# 2. Ghi một List các chuỗi vào my_notes.txt
notes_file_name = "my_notes.txt"
list_of_lines = [
    "Dòng ghi chú số 1.\n",
    "Dòng ghi chú số 2 với một vài thông tin.\n",
    "Dòng cuối cùng trong file ghi chú.\n"
]

try:
    with open(notes_file_name, "w", encoding="utf-8") as file:
        file.writelines(list_of_lines)
    print(f"Đã ghi danh sách các dòng vào tệp '{notes_file_name}'.")
except IOError as e:
    print(f"Lỗi khi ghi tệp '{notes_file_name}': {e}")
```

10.4.3. Thêm nội dung vào cuối một tệp hiện có, kết hợp try-except và with open()

Mục tiêu: Thực hành chế độ append và xử lý lỗi.

Bài tập: Tạo file append_to_file.py.

1. Đảm bảo có một tệp tên là log.txt (bạn có thể tạo thủ công hoặc dùng code để tạo rỗng).
2. Viết một hàm append_log(message) nhận vào một chuỗi message. Hàm này sẽ mở tệp log.txt ở chế độ 'a' (append), ghi message kèm theo dấu thời gian (sử dụng module datetime) và ký tự xuống dòng vào cuối tệp.
3. Sử dụng try-except để bắt các lỗi có thể xảy ra khi mở hoặc ghi tệp.
4. Gọi hàm append_log() nhiều lần với các thông điệp khác nhau.
5. Đọc lại nội dung của log.txt để kiểm tra.

Gợi ý:

```
# append_to_file.py
```

```
import datetime

print("--- THÊM NỘI DUNG VÀO CUỐI TẬP (APPEND) ---")

log_file_name = "application_log.txt"

# Bước 1: Đảm bảo file log.txt tồn tại (tạo rỗng nếu chưa có)
try:
    with open(log_file_name, "a", encoding="utf-8") as f:
        # Chỉ mở để đảm bảo file tồn tại, không ghi gì lúc này
        pass
    print(f"Đã đảm bảo tệp '{log_file_name}' tồn tại.")
except Exception as e:
    print(f"Không thể tạo/mở tệp '{log_file_name}': {e}")

def append_log(message):
    """
    Thêm một thông điệp log vào cuối tệp log_file_name với dấu thời gian.
    """
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    log_entry = f"[{timestamp}] {message}\n"

    try:
        with open(log_file_name, "a", encoding="utf-8") as file:
            file.write(log_entry)
        print(f"Đã ghi log: '{message}'")
    except IOError as e:
        print(f"Lỗi I/O khi ghi log vào tệp '{log_file_name}': {e}")
    except Exception as e:
        print(f"Đã xảy ra lỗi không mong muốn khi ghi log: {e}")

# Bước 4: Gọi hàm append_log() nhiều lần
append_log("Chương trình khởi động.")
append_log("Người dùng 'admin' đã đăng nhập.")
append_log("Đã xử lý dữ liệu X thành công.")
append_log("Lỗi: Không tìm thấy tài nguyên Y.")
append_log("Chương trình kết thúc.")

# Bước 5: Đọc lại nội dung của log.txt để kiểm tra
```



```
print(f"\n--- Nội dung tệp '{log_file_name}' sau khi ghi ---")
try:
    with open(log_file_name, "r", encoding="utf-8") as file:
        log_content = file.read()
        print(log_content)
except FileNotFoundError:
    print(f"Lỗi: Tệp '{log_file_name}' không tìm thấy sau khi ghi.")
```

10.5. Đọc/Ghi dữ liệu JSON & CSV

10.5.1. Lưu trữ một dict hoặc list của dict vào file JSON

Mục tiêu: Thực hành ghi dữ liệu Python sang JSON.

Bài tập: Tạo file json_rw.py.

1. Tạo một Dictionary Python đại diện cho thông tin một người (ví dụ: name, age, city, is_student).
2. Sử dụng json.dump() để lưu Dictionary này vào một tệp JSON có tên person.json. Đảm bảo tệp JSON được định dạng đẹp (sử dụng indent=4) và hỗ trợ ký tự tiếng Việt (ensure_ascii=False).
3. Tạo một List chứa nhiều Dictionary, mỗi Dictionary đại diện cho thông tin một sản phẩm (ví dụ: id, name, price, ``quantity``).
4. Lưu List này vào một tệp JSON khác tên là products.json với cùng định dạng.

Gợi ý:

```
# json_rw.py

import json

print("--- LƯU TRỮ DỮ LIỆU PYTHON VÀO FILE JSON ---")

# 1. Lưu trữ một Dictionary vào file JSON
person_data = {
    "name": "Lê Văn Tám",
    "age": 28,
    "city": "Hồ Chí Minh",
    "is_student": False,
    "hobbies": ["đọc sách", "chơi game", "nấu ăn"]
}
```

```
person_file = "person.json"
try:
    with open(person_file, "w", encoding="utf-8") as f:
        json.dump(person_data, f, indent=4, ensure_ascii=False)
        print(f"Đã lưu thông tin cá nhân vào '{person_file}'.")
except IOError as e:
    print(f"Lỗi khi lưu '{person_file}': {e}")

# 2. Lưu trữ một List các Dictionary vào file JSON
products_data = [
    {"id": "P001", "name": "Laptop Dell XPS", "price": 1500.00,
    "quantity": 5},
    {"id": "P002", "name": "Chuột Logitech", "price": 35.50, "quantity":
    100},
    {"id": "P003", "name": "Bàn phím cơ", "price": 120.00, "quantity":
    20},
    {"id": "P004", "name": "Màn hình 27 inch", "price": 299.99,
    "quantity": 15}
]

products_file = "products.json"
try:
    with open(products_file, "w", encoding="utf-8") as f:
        json.dump(products_data, f, indent=4, ensure_ascii=False)
        print(f"Đã lưu danh sách sản phẩm vào '{products_file}'.")
except IOError as e:
    print(f"Lỗi khi lưu '{products_file}': {e}")
```

10.5.2. Đọc dữ liệu từ file JSON và chuyển đổi thành cấu trúc Python

Mục tiêu: Thực hành đọc dữ liệu JSON vào Python.

Bài tập tiếp theo trong json_rw.py:

1. Đọc lại dữ liệu từ person.json và in ra tên và tuổi của người đó.
2. Đọc dữ liệu từ products.json. Duyệt qua List các sản phẩm và in ra tên và giá của từng sản phẩm.
3. Xử lý lỗi FileNotFoundError nếu các tệp JSON không tồn tại.

Gợi ý:

```
# json_rw.py (tiếp tục)
```

```
print("\n--- ĐỌC DỮ LIỆU TỪ FILE JSON VÀO PYTHON ---")

# 1. Đọc dữ liệu từ person.json
try:
    with open(person_file, "r", encoding="utf-8") as f:
        loaded_person = json.load(f)
        print(f"\nThông tin cá nhân đọc từ '{person_file}':")
        print(f"Tên: {loaded_person['name']}")
        print(f"Tuổi: {loaded_person['age']}")
except FileNotFoundError:
    print(f"Lỗi: Tệp '{person_file}' không tìm thấy.")
except json.JSONDecodeError:
    print(f"Lỗi: Tệp '{person_file}' không phải là định dạng JSON hợp lệ.")
except Exception as e:
    print(f"Đã xảy ra lỗi khi đọc '{person_file}': {e}")

# 2. Đọc dữ liệu từ products.json
try:
    with open(products_file, "r", encoding="utf-8") as f:
        loaded_products = json.load(f)
        print(f"\nDanh sách sản phẩm đọc từ '{products_file}':")
        for product in loaded_products:
            print(f"    - {product['name']} (Giá: {product['price']:.2f} VNĐ)")
except FileNotFoundError:
    print(f"Lỗi: Tệp '{products_file}' không tìm thấy.")
except json.JSONDecodeError:
    print(f"Lỗi: Tệp '{products_file}' không phải là định dạng JSON hợp lệ.")
except Exception as e:
    print(f"Đã xảy ra lỗi khi đọc '{products_file}': {e}")
```

10.5.3. Đọc dữ liệu đơn giản từ file CSV bằng module csv hoặc đọc thủ công

Mục tiêu: Thực hành đọc dữ liệu từ CSV.

Bài tập: Tạo file csv_rw.py.

1. Tạo một tệp CSV mới tên là students.csv với ít nhất 3 dòng dữ liệu, bao gồm tiêu đề.

- Nội dung mẫu students.csv:

```
ID,Name,Age,Major
1,Bim,20,Computer Science
2,Bo,21,Electrical Engineering
3,Bin,19,Business Administration
```

2. Sử dụng `csv.reader` để đọc tệp `students.csv`. In ra tiêu đề và sau đó in ra thông tin của từng sinh viên dưới dạng một List.
3. (Tùy chọn) Thử đọc thủ công: Đọc từng dòng bằng `readline()` hoặc lặp qua tệp, sau đó dùng `split(',')` để tách các giá trị.

Gợi ý:

```
# csv_rw.py

import csv

print("--- ĐỌC DỮ LIỆU TỪ FILE CSV ---")

students_csv_file = "students.csv"

# Bước 1: Tạo file students.csv để test
try:
    with open(students_csv_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["ID", "Name", "Age", "Major"])
        writer.writerow(["1", "Bim", "20", "Computer Science"])
        writer.writerow(["2", "Bo", "21", "Electrical Engineering"])
        writer.writerow(["3", "Bin", "19", "Business Administration"])
    print(f"Đã tạo/ghi đè file '{students_csv_file}' để kiểm tra.")
except Exception as e:
    print(f"Không thể tạo file {students_csv_file}: {e}")

# 2. Đọc dữ liệu từ students.csv bằng module csv.reader
print(f"\n--- Đọc '{students_csv_file}' bằng csv.reader ---")
try:
    with open(students_csv_file, "r", newline="", encoding="utf-8") as file:
        reader = csv.reader(file)
        header = next(reader) # Đọc dòng tiêu đề
        print(f"Tiêu đề: {header}")
        print("Dữ liệu sinh viên:")
```

```
        for row in reader:
            print(row) # Mỗi row là một list các chuỗi
except FileNotFoundError:
    print(f"Lỗi: Tệp '{students_csv_file}' không tìm thấy.")
except Exception as e:
    print(f"Đã xảy ra lỗi khi đọc CSV: {e}")

# 3. (Tùy chọn) Đọc dữ liệu từ file CSV thủ công (không dùng module csv)
print(f"\n--- Đọc '{students_csv_file}' thủ công (dùng split(', ')) ---")
try:
    with open(students_csv_file, "r", encoding="utf-8") as file:
        # Đọc tiêu đề
        header_line = file.readline().strip()
        manual_header = header_line.split(', ')
        print(f"Tiêu đề (thủ công): {manual_header}")

        # Đọc từng dòng dữ liệu
        print("Dữ liệu sinh viên (thủ công):")
        for line in file:
            data = line.strip().split(', ')
            print(data)
except FileNotFoundError:
    print(f"Lỗi: Tệp '{students_csv_file}' không tìm thấy.")
except Exception as e:
    print(f"Đã xảy ra lỗi khi đọc CSV thủ công: {e}")
```

10.6. Tổng hợp & Ôn tập (Mini-Project 1)

Áp dụng các kiến thức về biến, cấu trúc dữ liệu (List, Dict), vòng lặp, điều kiện, hàm, xử lý chuỗi và tệp để xây dựng một mini-project nhỏ (ví dụ: chương trình quản lý danh bạ đơn giản lưu trong file TXT/JSON)

Mục tiêu: Tổng hợp các kiến thức đã học vào một ứng dụng thực tế nhỏ.

Bài tập: Tạo file `contact_manager.py`.

Xây dựng chương trình quản lý danh bạ đơn giản:

- Chương trình cho phép người dùng:
 1. **Thêm liên hệ mới:** Nhập tên, số điện thoại, email (tùy chọn).
 2. **Xem tất cả liên hệ:** Hiển thị danh sách tất cả các liên hệ.
 3. **Tìm kiếm liên hệ:** Tìm theo tên, hiển thị thông tin chi tiết.

4. **Xóa liên hệ:** Xóa một liên hệ theo tên.

5. **Thoát.**

- **Lưu trữ dữ liệu:** Dữ liệu danh bạ sẽ được lưu trữ trong một tệp JSON (ví dụ: contacts.json).
 - Mỗi liên hệ là một Dictionary (ví dụ: {"name": "Bim", "phone": "12345", "email": "a@example.com"}).
 - Toàn bộ danh bạ sẽ là một List các Dictionary.
- Khi chương trình khởi động, đọc dữ liệu từ contacts.json. Nếu tệp không tồn tại, khởi tạo một danh bạ rỗng.
- Trước khi thoát chương trình, lưu toàn bộ danh bạ trở lại contacts.json.
- Sử dụng hàm, vòng lặp, điều kiện, và xử lý lỗi try-except phù hợp.

Gợi ý: Thực hành viết code sạch, có comment và tổ chức thành hàm.

```
# contact_manager.py

import json
import os # Dùng để kiểm tra sự tồn tại của file

CONTACTS_FILE = "contacts.json"
contacts = [] # Biến toàn cục để lưu danh sách liên hệ

def load_contacts():
    """Tải danh sách liên hệ từ file JSON."""
    global contacts
    if os.path.exists(CONTACTS_FILE):
        try:
            with open(CONTACTS_FILE, "r", encoding="utf-8") as f:
                contacts = json.load(f)
            print("Đã tải danh bạ thành công.")
        except json.JSONDecodeError:
            print("Lỗi đọc file JSON. Tạo danh bạ rỗng.")
            contacts = []
        except Exception as e:
            print(f"Lỗi khi tải danh bạ: {e}. Tạo danh bạ rỗng.")
            contacts = []
    else:
        print("File danh bạ không tồn tại. Tạo danh bạ rỗng.")
```

```
contacts = []

def save_contacts():
    """Lưu danh sách liên hệ vào file JSON."""
    try:
        with open(CONTACTS_FILE, "w", encoding="utf-8") as f:
            json.dump(contacts, f, indent=4, ensure_ascii=False)
        print("Đã lưu danh bạ thành công.")
    except IOError as e:
        print(f"Lỗi khi lưu danh bạ: {e}")
    except Exception as e:
        print(f"Lỗi không mong muốn khi lưu danh bạ: {e}")

def add_contact():
    """Thêm một liên hệ mới vào danh bạ."""
    name = input("Nhập tên liên hệ: ").strip()
    if not name:
        print("Tên không được để trống.")
        return

    # Kiểm tra tên trùng lặp (có thể cải tiến để cho phép nhiều người cùng tên)
    for contact in contacts:
        if contact['name'].lower() == name.lower():
            print(f"Liên hệ với tên '{name}' đã tồn tại.")
            return

    phone = input("Nhập số điện thoại: ").strip()
    email = input("Nhập email (tùy chọn): ").strip()

    new_contact = {
        "name": name,
        "phone": phone,
        "email": email if email else None # Lưu None nếu email trống
    }
    contacts.append(new_contact)
    print(f"Đã thêm liên hệ '{name}'.")

def view_contacts():
    """Hiển thị tất cả liên hệ trong danh bạ."""
```

```
    if not contacts:
        print("Danh bạ trống.")
        return

    print("\n--- DANH SÁCH LIÊN HỆ ---")
    for i, contact in enumerate(contacts):
        email_info = f", Email: {contact['email']}" if contact['email']
    else ""
        print(f"{i+1}. Tên: {contact['name']], SĐT: {contact['phone']}{email_info}")
        print("-----")

def search_contact():
    """Tìm kiếm liên hệ theo tên."""
    search_name = input("Nhập tên liên hệ cần tìm: ").strip().lower()
    found_contacts = [contact for contact in contacts if search_name in
contact['name'].lower()]

    if not found_contacts:
        print(f"Không tìm thấy liên hệ nào với tên '{search_name}'.")
        return

    print(f"\n--- KẾT QUẢ TÌM KIẾM CHO '{search_name}' ---")
    for contact in found_contacts:
        email_info = f", Email: {contact['email']}" if contact['email']
    else ""
        print(f"Tên: {contact['name']], SĐT: {contact['phone']}{email_info}")
        print("-----")

def delete_contact():
    """Xóa liên hệ theo tên."""
    name_to_delete = input("Nhập tên liên hệ cần xóa: ").strip()

    initial_len = len(contacts)
    # Lọc ra các liên hệ không khớp với tên cần xóa (case-insensitive)
    contacts[:] = [contact for contact in contacts if
contact['name'].lower() != name_to_delete.lower()]

    if len(contacts) < initial_len:
        print(f"Đã xóa liên hệ '{name_to_delete}' (nếu tồn tại).")
```



```
        else:
            print(f"Không tìm thấy liên hệ '{name_to_delete}' để xóa.")

def display_menu():
    """Hiển thị menu chính."""
    print("\n--- CHƯƠNG TRÌNH QUẢN LÝ DANH BẠ ---")
    print("1. Thêm liên hệ mới")
    print("2. Xem tất cả liên hệ")
    print("3. Tìm kiếm liên hệ")
    print("4. Xóa liên hệ")
    print("5. Thoát")
    print("-----")

def main():
    """Hàm chính điều khiển chương trình."""
    load_contacts() # Tải danh bạ khi khởi động

    while True:
        display_menu()
        choice = input("Nhập lựa chọn của bạn (1-5): ")

        if choice == '1':
            add_contact()
        elif choice == '2':
            view_contacts()
        elif choice == '3':
            search_contact()
        elif choice == '4':
            delete_contact()
        elif choice == '5':
            save_contacts() # Lưu danh bạ trước khi thoát
            print("Đang thoát chương trình. Tạm biệt!")
            break
        else:
            print("Lựa chọn không hợp lệ. Vui lòng thử lại.")

if __name__ == "__main__":
    main()
```